# Laboratorio 3 — Factorización matricial con SVD para sistemas de recomendación

Alvaro Andres Castiblanco Lopez[1], Bryan Mauricio Guzmán García[2], Juan Carlos Tovar Orjuela[3] y Lorraine Jazlady Rojas Parra[4]

[1]Maestría en Ingeniería de la Información, E-mail: a.castiblancol@uniandes.edu.co.
[2]Maestría en Arquitectura de Tecnologías de Información, E-mail: b.guzmang@uniandes.edu.co.
[3]Maestría en Ingeniería de la Información, E-mail: jc.tovaro1@uniandes.edu.co.
[4]Maestría en Ingeniería de la Información, E-mail: lj.rojasp1@uniandes.edu.co.

## 1. Introducción

Este informe documenta el desarrollo del **Laboratorio 3** del curso de Sistemas de Recomendación, centrado en el uso de **Descomposición en Valores Singulares (SVD)** para modelar preferencias usuario–ítem. El objetivo del laboratorio fue conectar la intuición algebraica de la factorización matricial con su aplicación práctica en recomendación, usando tanto una implementación especializada para ratings dispersos (`Surprise`) como una aproximación general con `TruncatedSVD` de `scikit-learn`.

De manera específica, el trabajo cubrió: (i) la exploración de los conjuntos de entrenamiento, validación y prueba; (ii) la interpretación de la dispersión de la matriz usuario–ítem y de la distribución de ratings; (iii) la comprensión conceptual de SVD mediante una matriz pequeña controlada; (iv) el entrenamiento de un modelo SVD con sesgos usando `Surprise`; (v) la reconstrucción de la matriz de predicciones y el análisis del error absoluto sobre coordenadas conocidas; (vi) la evaluación del modelo con RMSE en train, validación y test; (vii) la selección de hiperparámetros; y (viii) una comparación con `TruncatedSVD` como método genérico de reducción de dimensionalidad.

### *1.1. Flujo general del laboratorio*

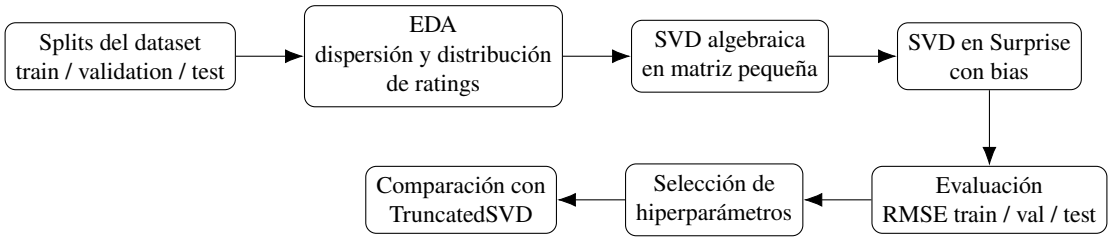


Figure 1: Pipeline seguido en el laboratorio.

## 2. Exploración de los datasets de entrenamiento, validación y prueba

El laboratorio trabajó con tres particiones del conjunto de datos: entrenamiento, validación y prueba. La Tabla 1 resume sus tamaños en número de ratings, usuarios e ítems distintos.

Tabla 1: Resumen de los conjuntos de entrenamiento, validación y prueba.

| Split | # ratings | # usuarios | # ítems |
|---|---|---|---|
| Train | 81 140 | 911 | 1 672 |
| Validation | 9 430 | 943 | 1 129 |
| Test | 9 430 | 943 | 1 145 |

### *2.1. Dispersión de la matriz usuario–ítem*

La matriz usuario–ítem en entrenamiento tiene dimensión $911 \times 1672$, es decir, un total de 1 523 192 posibles entradas. Dado que solo se observaron 81 140 ratings, la dispersión se calculó como:

$$\text{sparsity} = 1 - \frac{81\,140}{911 \times 1672} \approx 0.9467$$

Esto equivale a una dispersión de **94.6720%**. En otras palabras, casi el 95% de la matriz está vacía. Este resultado es completamente consistente con escenarios reales de recomendación: los usuarios interactúan con una fracción pequeña del catálogo, por lo que el aprendizaje debe hacerse a partir de observaciones escasas y muy dispersas.

### *2.2. Distribución de ratings*

Se visualizó la distribución de ratings en los tres conjuntos mediante gráficos de barras. El comportamiento observado fue similar entre entrenamiento, validación y test, lo cual es deseable porque sugiere consistencia entre particiones. En términos generales, la distribución mostró un sesgo hacia calificaciones medias y altas, lo que indica una mayor densidad de evidencia positiva que negativa.

Esta observación es importante porque influye directamente en el promedio global del modelo y en la interpretación de los sesgos. Si predominan ratings altos, el recomendador tenderá inicialmente a predecir valores relativamente elevados, salvo que los sesgos de usuario e ítem y la interacción latente corrijan esa tendencia. En otras palabras, el dataset contiene más información sobre qué gusta que sobre qué desagrada.

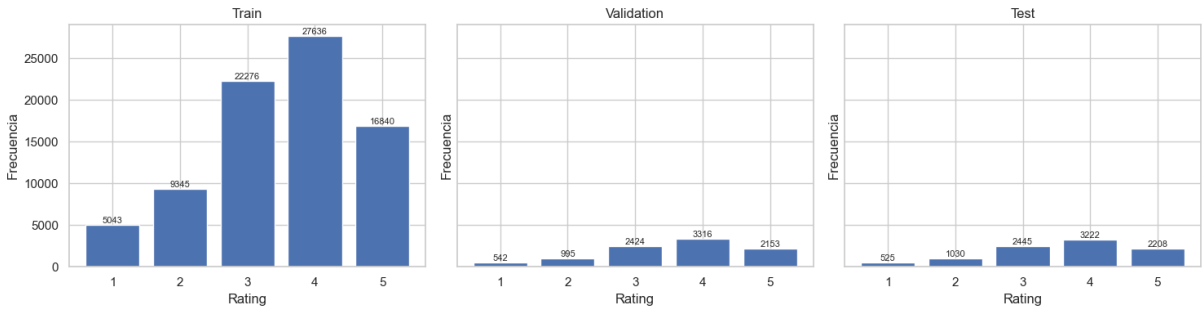


Figure 2: Distribución de ratings en los conjuntos de train, validación y test.

## 3. Entrenamiento de un modelo SVD con *bias*

En recomendación, la factorización matricial no se realiza sobre una matriz densa completa. En cambio, se aprenden factores latentes únicamente a partir de ratings observados. El modelo SVD usado en `Surprise` estima la predicción mediante:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u q_i^T$$

where $\mu$ es el promedio global, $b_u$ y $b_i$ son los sesgos de usuario e ítem, y $p_u$, $q_i$ son sus vectores latentes. Para el entrenamiento inicial se usaron los hiperparámetros:

- `n_factors = 20`
- `n_epochs = 60`
- `biased = True`
- `lr_all = 0.005`
- `reg_all = 0.02`
- `random_state = 42`

### *3.1. Parámetros aprendidos*

Tras el entrenamiento, el modelo almacenó:

- un promedio global $\mu$;
- una matriz $P$ de factores latentes para usuarios de dimensión $(911, 20)$;
- una matriz $Q$ de factores latentes para ítems de dimensión $(1672, 20)$;
- un vector de sesgos de usuario $b_u$ de dimensión $(911, )$;
- un vector de sesgos de ítem $b_i$ de dimensión $(1672, )$.

Estos tamaños confirman que cada usuario y cada ítem queda representado en un espacio latente de 20 dimensiones. El efecto principal de esta factorización es condensar la información dispersa de la matriz original en vectores compactos que modelan afinidades implícitas. A su vez, la inclusión de sesgos permite que patrones sistemáticos simples —como usuarios que siempre califican alto o películas que son generalmente bien evaluadas— no recaigan sobre los factores latentes, liberando capacidad del modelo para aprender relaciones más complejas.

### *3.2. Distribución de sesgos de usuarios e ítems*

Como parte de la inspección del modelo entrenado, se analizaron los sesgos aprendidos para usuarios ($b_u$) e ítems ($b_i$). Estos sesgos capturan tendencias sistemáticas que no dependen de las afinidades latentes: en particular, si un usuario tiende a calificar más alto o más bajo que el promedio global, o si una película suele recibir mejores o peores valoraciones de manera general.

La Figura 3 muestra la distribución de ambos conjuntos de sesgos. En general, se espera que la mayor parte de los valores se concentre alrededor de cero, con algunos usuarios e ítems alejándose de ese centro. Un valor positivo de $b_u$ indica un usuario más generoso al calificar, mientras que un valor negativo indica mayor severidad. De forma análoga, un valor positivo de $b_i$ sugiere que el ítem tiende a recibir calificaciones altas de manera sistemática.

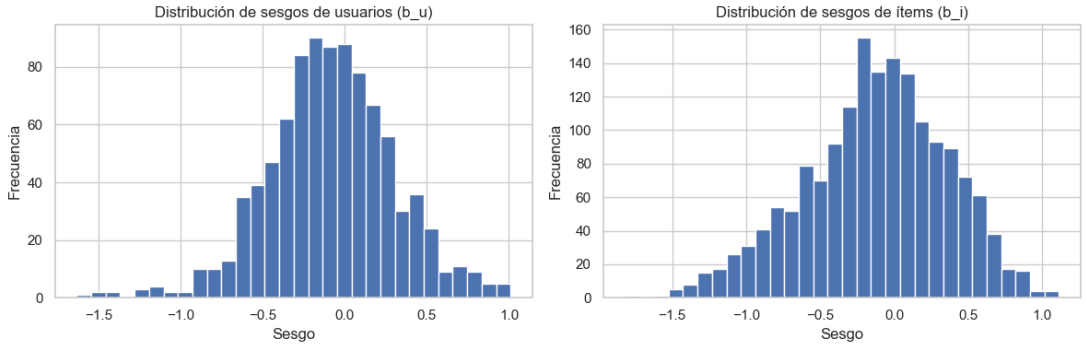


Figure 3: Distribución de los sesgos aprendidos para usuarios ($b_u$) e ítems ($b_i$).

Adicionalmente, se identificó el ítem con mayor sesgo positivo, correspondiente a la película *Waiting for Guffman (1996)*, con identificador `1449`. Esto indica que, independientemente de las afinidades latentes específicas, el modelo considera que dicho ítem tiende a recibir valoraciones particularmente altas.

### *3.3. Verificación manual de la fórmula de predicción*

Se tomó la primera fila de `train_df` y se comparó la predicción calculada manualmente con la salida de `algo.predict(user, item).est`. Ambos valores coincidieron, confirmando que la librería implementa exactamente la expresión:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u q_i^T$$

Esto valida tanto la interpretación teórica del modelo como la correcta extracción de sus parámetros internos.

### *3.4. Reconstrucción de la matriz de ratings*

A partir de los parámetros aprendidos se reconstruyó la matriz completa de predicciones según:

$$\hat{R} = \mu + b_u \mathbf{1}^T + \mathbf{1} b_i^T + PQ^T$$

Posteriormente, los valores se recortaron al rango $[1, 5]$ para respetar la escala original de ratings. La matriz reconstruida tuvo forma $(911, 1672)$, consistente con la matriz usuario–ítem de entrenamiento.

### *3.5. ¿Qué puede decir sobre la matriz original de ratings de entrenamiento?*

La matriz original de entrenamiento es **altamente dispersa**, con ratings observados en una pequeña fracción de las coordenadas posibles. Su principal característica es que contiene información real únicamente en las entradas conocidas, mientras que la inmensa mayoría de posiciones están vacías. Esto implica que el modelo debe inferir la estructura de preferencias a partir de un subconjunto limitado de observaciones, apoyándose en regularidades globales: promedio global, sesgos y patrones latentes compartidos.

Otra observación importante es que esta matriz no puede interpretarse como una matriz densa tradicional de álgebra lineal. Los valores faltantes no significan "cero gusto", sino "sin observación". Por eso, una implementación especializada como `Surprise` resulta adecuada: aprende exclusivamente desde ratings observados y evita confundir ausencia de interacción con evidencia negativa.

### *3.6. ¿Qué puede decir sobre el error absoluto en las coordenadas conocidas?*

Al comparar la matriz original con la reconstruida únicamente en las coordenadas conocidas, se observó que el error absoluto no es uniforme: algunas zonas presentan diferencias pequeñas y otras mayores. Esto es esperable, ya que el modelo intenta representar una gran cantidad de interacciones mediante un número finito de factores latentes. En general, el mapa de calor del error permite ver qué regiones del espacio usuario–ítem quedan mejor explicadas por el modelo y cuáles son más difíciles de ajustar.

Conceptualmente, el error absoluto en entradas conocidas representa cuánto se desvía la predicción reconstruida del rating real observado. Si en promedio estos errores son moderados, ello indica que la combinación de sesgos y factores latentes logra capturar la estructura dominante del dataset. A la vez, la existencia de errores locales más altos es una evidencia natural de que el modelo no memoriza exactamente todas las observaciones, especialmente bajo regularización.

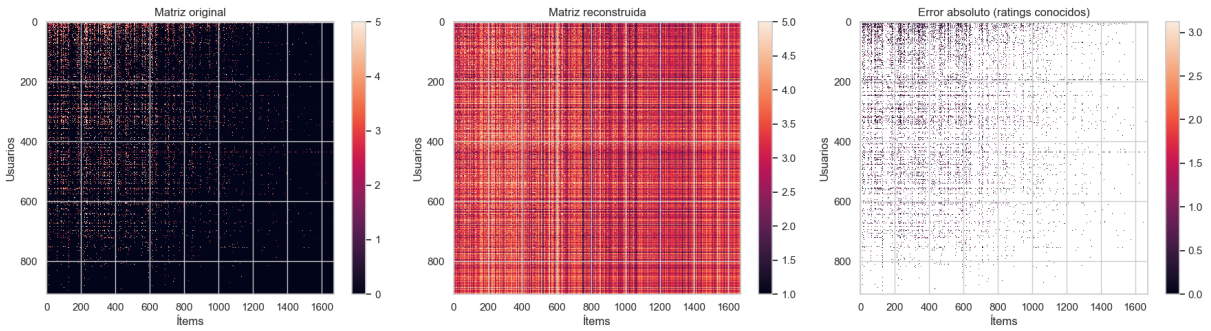


Figure 4: Matriz original, matriz reconstruida y error absoluto sobre coordenadas conocidas.

## 4. Evaluación de las predicciones

La calidad del modelo se midió mediante RMSE sobre entrenamiento, validación y test. Los resultados obtenidos fueron los siguientes:

Tabla 2: RMSE del modelo SVD con bias entrenado con los hiperparámetros iniciales.

| Conjunto | RMSE |
|---|---|
| Train | 0.6314 |
| Validation | 0.9905 |
| Test | 1.0062 |

### *4.1. ¿Qué puede decir del modelo SVD entrenado con los hiperparámetros suministrados?*

El modelo logra un ajuste muy fuerte sobre entrenamiento, con un RMSE de 0.6314, pero su desempeño en validación y test empeora de manera considerable, ubicándose alrededor de 1.0. Esto constituye **evidencia de sobreajuste**. Es decir, el modelo aprendió muy bien las interacciones observadas en train, pero no generaliza con la misma calidad a datos no vistos.

Al mismo tiempo, el hecho de que validación (0.9905) y test (1.0062) sean muy cercanos entre sí es una señal de consistencia: el comportamiento fuera de entrenamiento es estable. Por tanto, el problema principal no es una evaluación ruidosa o inestable, sino una brecha clara entre el error optimista de entrenamiento y la capacidad real de generalización. En síntesis, el modelo base es funcional y captura estructura útil, pero con esta configuración todavía tiene una complejidad alta respecto a lo que los datos permiten generalizar.

## 5. Selección de hiper-parámetros

Para estudiar el comportamiento del modelo se exploraron dos hiperparámetros clave: el número de factores latentes $k$ y la regularización $\lambda$ (`reg_all`). El objetivo fue encontrar un mejor equilibrio entre capacidad de representación y control del sobreajuste.

### *5.1. Efecto de aumentar el número de dimensiones latentes $k$*

Se evaluaron valores de $k \in \{5, 10, 20, 40\}$ manteniendo fijos el resto de hiperparámetros. Conceptualmente, aumentar $k$ incrementa la capacidad del modelo para representar patrones más finos de interacción usuario–ítem. Esto suele reducir primero el error de entrenamiento, porque el modelo dispone de más grados de libertad para ajustarse a las observaciones conocidas.

Sin embargo, un incremento excesivo en $k$ puede deteriorar la generalización si la regularización no es suficiente. En ese escenario, la matriz de factores latentes comienza a absorber ruido o particularidades del conjunto de entrenamiento, aumentando la brecha entre train y validación. Por tanto, el valor óptimo de $k$ no es el que produce el menor RMSE en entrenamiento, sino el que minimiza el error en validación.

En términos de parámetros, aumentar $k$ expande las matrices $P$ y $Q$, incrementando el número total de parámetros libres del modelo. Con ello, la representación latente puede capturar relaciones más ricas, pero también crece el riesgo de sobreajuste. En el informe final se recomienda incluir una figura con curvas de RMSE (train, validación y test) en función de $k$ para evidenciar este comportamiento.

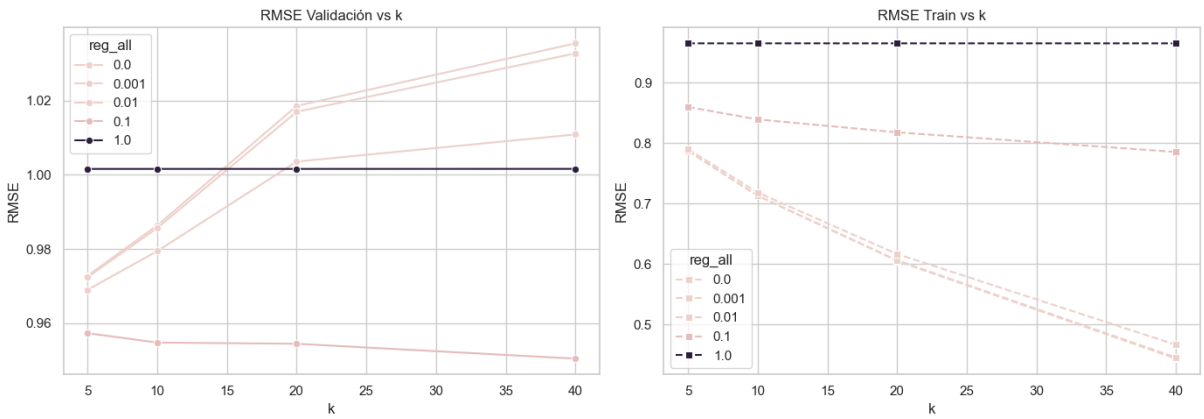


Figure 5: Efecto del número de factores latentes $k$ sobre los errores de train, validación y test.

### 5.2. *Efecto de la regularización para el mejor $k$*

Para el mejor valor de $k$ según validación, se analizó el efecto de variar la regularización en valores del tipo $[0.001, 0.01, 0.1, 1, 10]$. La regularización penaliza magnitudes grandes en factores y sesgos, por lo que actúa como mecanismo de control de complejidad. En general, cuando $\lambda$ es demasiado pequeño, el modelo tiene mayor libertad para ajustarse al train y aparece sobreajuste; cuando $\lambda$ es demasiado grande, los parámetros se contraen excesivamente y el modelo puede subajustar.

Sobre los parámetros aprendidos, el efecto esperado es claro: con mayor regularización, los vectores latentes y sesgos tienden a tener menores magnitudes. Esto reduce varianza y estabiliza las predicciones, pero si la penalización es excesiva el modelo pierde capacidad expresiva. Por ello, el mejor valor de regularización suele encontrarse en un punto intermedio, donde se logra disminuir la brecha train–validación sin destruir la señal útil contenida en los datos.

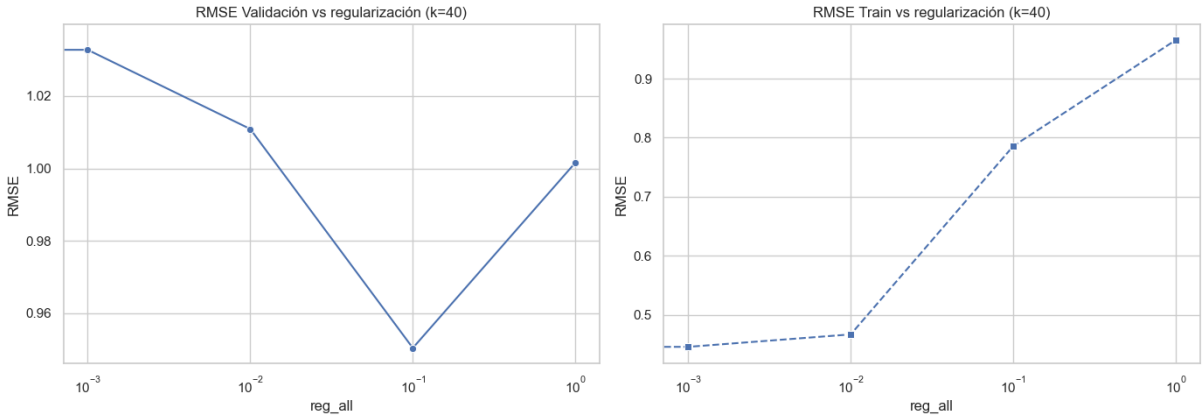


Figure 6: Efecto de la regularización sobre los errores de train, validación y test para el mejor valor de $k$.

### 5.3. *Mejor combinación encontrada*

La mejor combinación de hiperparámetros debe seleccionarse como aquella que minimiza el RMSE en validación. y se reporta con sus valores de train, validación y test.

Tabla 3: Mejor combinación de hiperparámetros según RMSE de validación.

| Hiperparámetro / métrica | Valor |
|---|---|
| $k$ óptimo | `40` |
| `reg_all` óptimo | `0.1` |
| RMSE train | `0.7857` |
| RMSE validation | `0.9504` |
| RMSE test | `0.9691` |

### *5.4. ¿Qué otros hiperparámetros puede configurar para mejorar el error en validación?*

Además de $k$ y la regularización, otro hiperparámetro importante es la **tasa de aprendizaje** (`lr_all`). Este parámetro controla qué tan grandes son las actualizaciones durante el entrenamiento por descenso estocástico. Si la tasa de aprendizaje es muy alta, el entrenamiento puede volverse inestable y no converger adecuadamente; si es muy baja, el modelo puede requerir muchas épocas para acercarse a una buena solución e incluso quedar subentrenado.

También podrían ajustarse hiperparámetros más específicos, como el número de épocas (`n_epochs`), o incluso tasas de aprendizaje y regularizaciones separadas para sesgos y factores latentes si se desea una calibración más fina. No obstante, en este laboratorio la tasa de aprendizaje es el siguiente candidato natural a explorar después de $k$ y `reg_all`, porque afecta directamente la convergencia y puede mejorar el RMSE de validación si se encuentra un valor más adecuado para la escala del problema.

## 6. Conclusiones

La principal conclusión del laboratorio es que la factorización matricial con SVD constituye una herramienta poderosa para modelar preferencias en sistemas de recomendación, especialmente cuando la matriz usuario–ítem es altamente dispersa. La formulación con sesgos permite separar tendencias globales simples del componente latente de afinidad, lo que mejora la interpretabilidad del modelo y hace un uso más eficiente de los factores aprendidos. Además, la reconstrucción de la matriz y la verificación manual de la fórmula de predicción ayudaron a conectar la teoría de SVD con su implementación práctica.

En términos de desempeño, el modelo SVD entrenado con los hiperparámetros iniciales mostró una clara capacidad para ajustarse a los datos de entrenamiento, pero también evidencia de sobreajuste al compararlo con validación y test. Esto resalta la importancia de la evaluación offline con múltiples particiones y del uso de un conjunto de validación para seleccionar hiperparámetros. En particular, el número de factores latentes y la regularización controlan un balance delicado entre capacidad del modelo y generalización.

Finalmente, la comparación con `TruncatedSVD` mostró que una descomposición genérica puede ser útil como ejercicio conceptual y como herramienta de reducción de dimensionalidad, pero no sustituye las ventajas de una librería especializada como `Surprise` cuando se trabaja con ratings dispersos. En recomendación, tratar faltantes como ceros centrados implica perder información semántica importante, ya que la ausencia de interacción no equivale a desinterés. Por ello, para problemas reales de predicción de ratings, una implementación especializada sigue siendo preferible.

## Referencias

[1] scikit-learn developers. *TruncatedSVD*. Documentación oficial de scikit-learn.
[2] F. Maxwell Harper and Joseph A. Konstan. *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems, 2015.